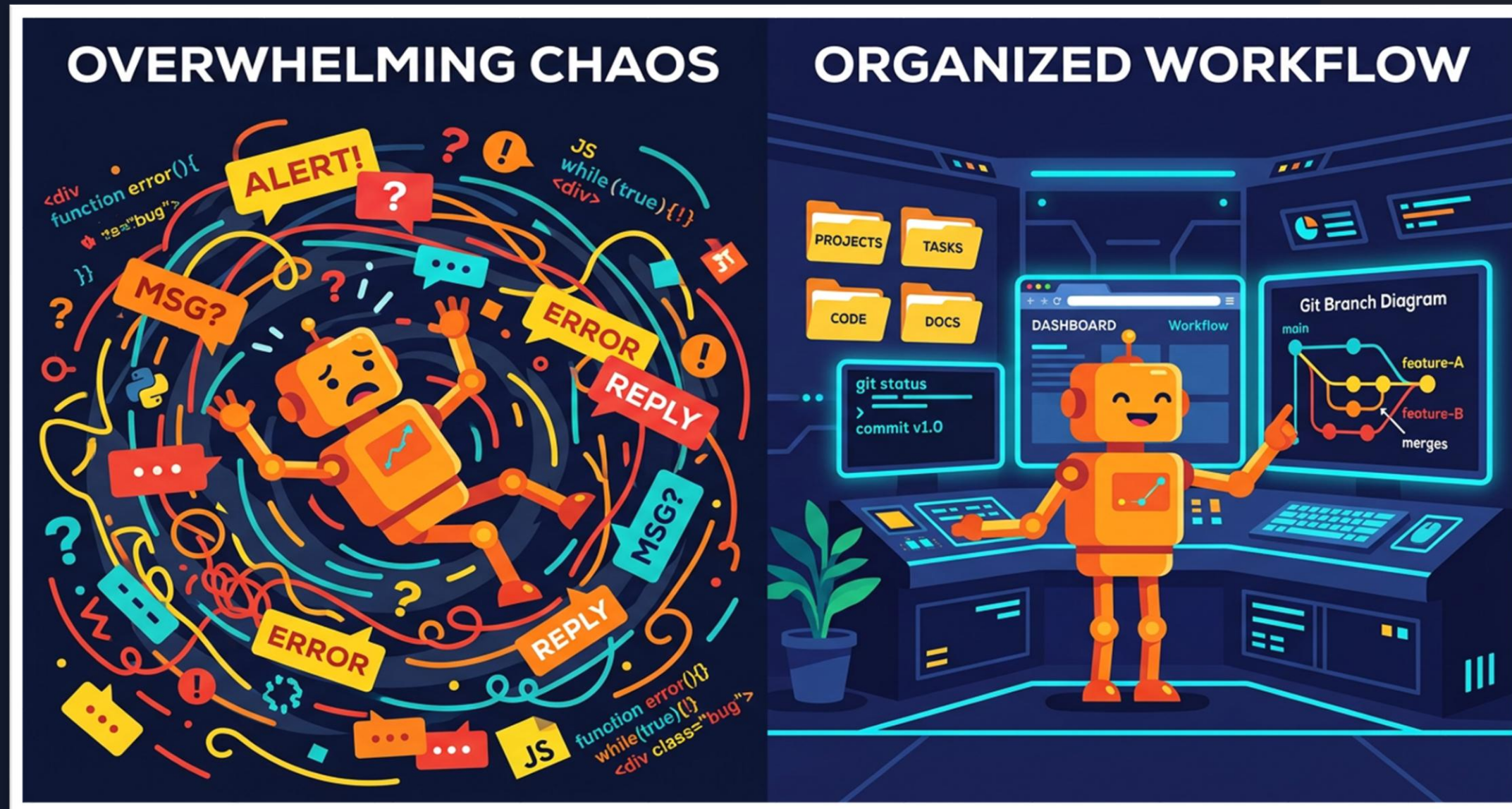


AI-Driven Development



Van vibe coding naar controleerbare AI-coding workflows

Vibe coding is **snel**. Maar wat **bewijst** het?

- Code genereren is makkelijk geworden
- Verifiëren blijft moeilijk
- Overdragen aan iemand anders is vaak nog moeilijker



Herkenbaar? De agent zegt **"done"**, jij bent **twee uur** kwijt.



Deze training gaat niet over "nog betere prompts". Ze gaat over **controle terugnemen**.

- Tests blijken niet gedraaid
- UI screenshot komt van de verkeerde runtime
- Agent heeft 30 files aangepast voor één bug
- Volgende sessie weet niet meer wat vorige deed



Onze case vandaag: **PersonaLab**



- AI-persona's zijn herkenbaar, actueel en risicovol
- Daarom zijn scope, toestemming en bewijs cruciaal
- We bouwen niet de hele app
- We volgen één slice: **BL-002 — Consent gate**



Van **losse prompts** naar een **workflow**

1 Losse prompt

2 Betere prompt

3 State files

4 Tools & verificatie

5 Proof bundle

6 CTO review

7 Team workflow



Het **Probleem**

Waarom AI coding agents vaak ontsporen



Context Decay

Beslissingen verdwijnen in chat history. De agent vergeet. Nieuwe sessies starten blind.

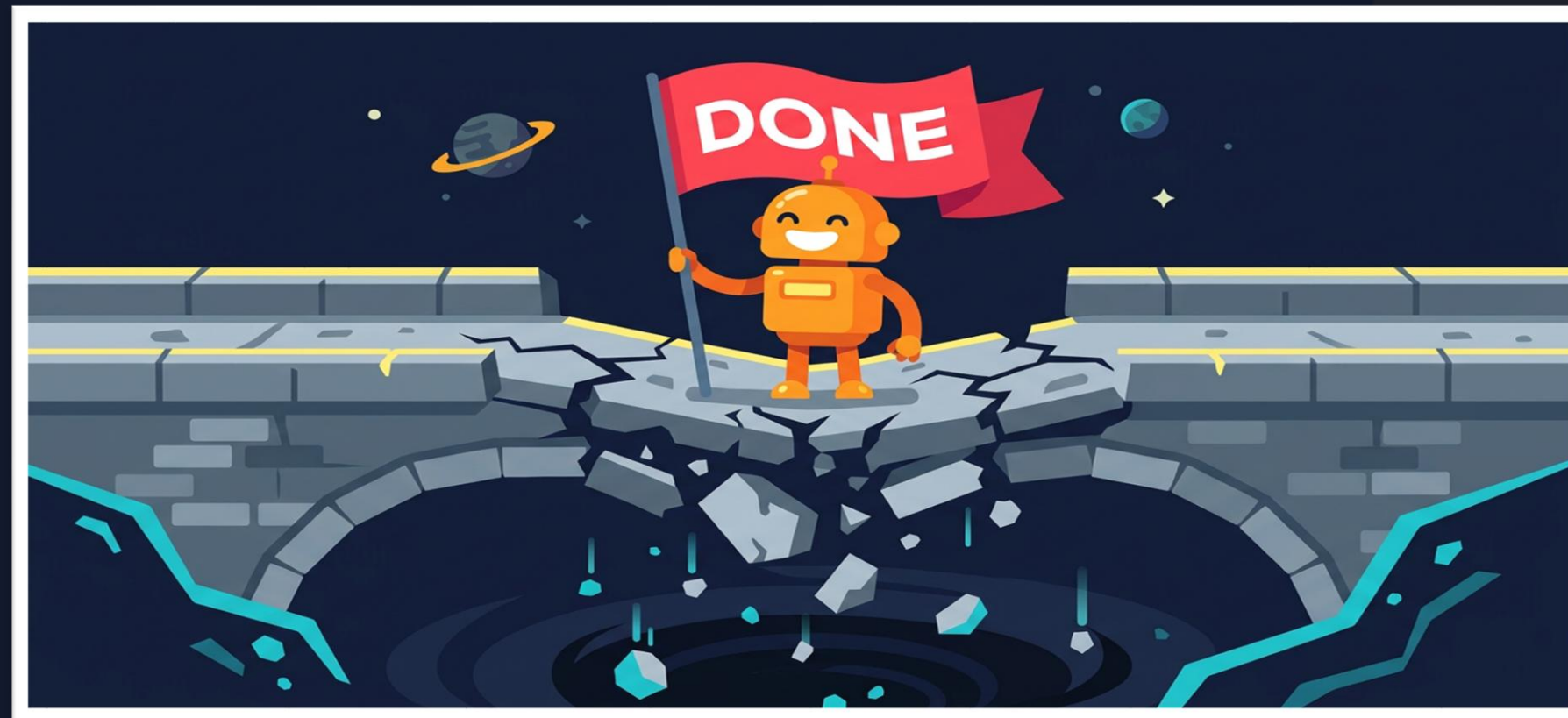
- Na ~10-20 prompts raakt de agent het overzicht kwijt
- Belangrijke aannames verdwijnen
- De agent beschrijft features die niet bestaan



Fake Progress

"Implemented" zonder werkende code. "Tested" zonder testoutput. Screenshot zonder runtime identity.

- Agent claimt "klaar" zonder te controleren
- Over-eager refactoring: 50 files gewijzigd voor 1 regel functie
- Stilste gevaar: het ziet eruit als vooruitgang



Het echte probleem is **ontbrekende projectwaarheid**

De agent liegt meestal niet bewust. Hij werkt gewoon met onvolledige waarheid.

- Chat is vluchtig
- Repo-state is vaak onduidelijk
- Runtime kan afwijken van code
- "Klaar" betekent niets zonder bewijs



02.

StateDD als **Project-OS**

De repo als bron van waarheid



De repo als **bron van waarheid**

GUIDING THE AGENTS



Chat is voor communicatie. De repo is voor waarheid.

- Persistent — leeft na elke sessie
- Versioned via git
- Iedereen leest dezelfde truth
- Machine-readable via YAML

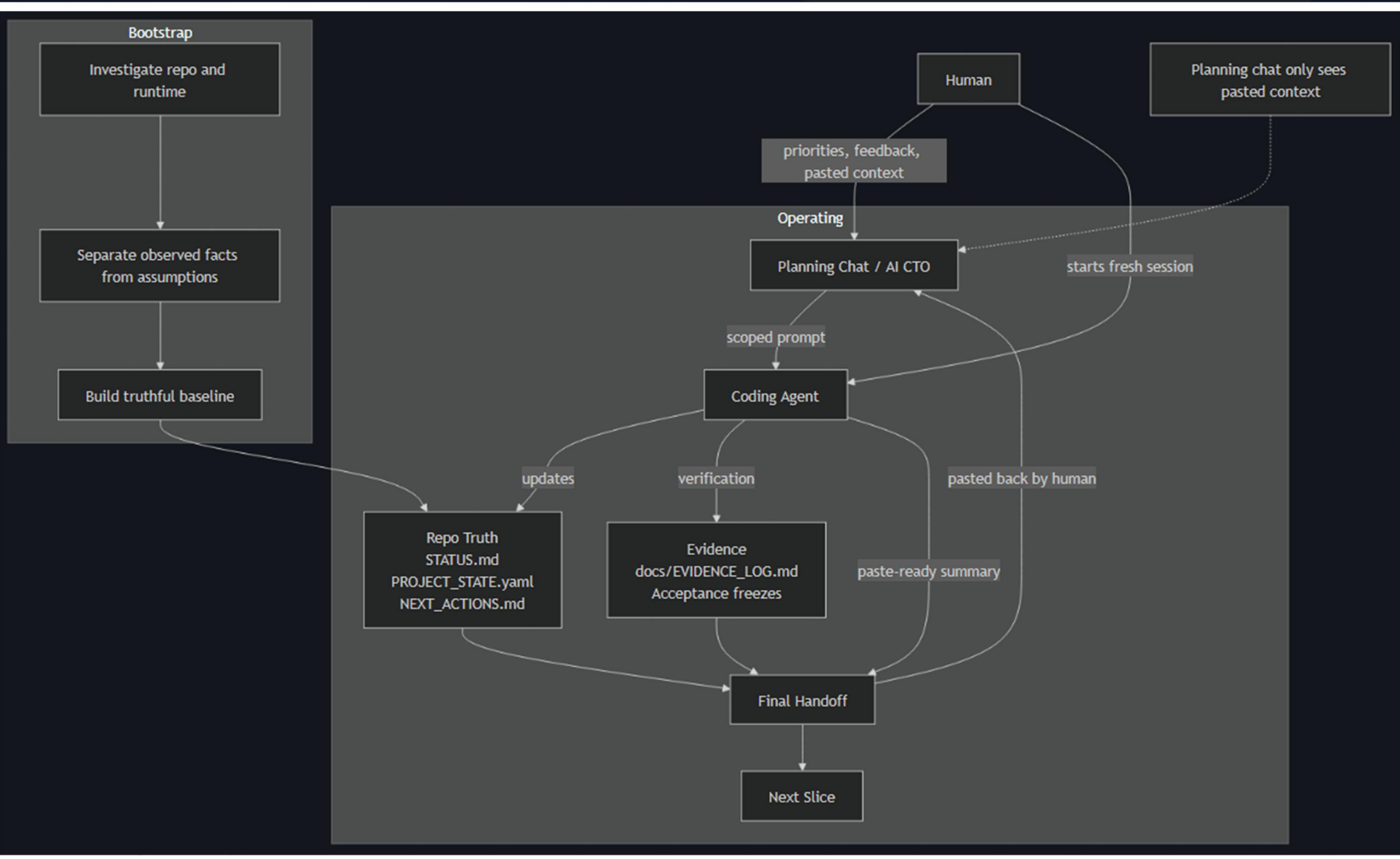


De StateDD Control Loop



- Human/CTO kiest richting
- Coding agent voert uit
- Tools verifiëren
- Proof bundle bewaart bewijs
- CTO agent beoordeelt





De volledige StateDD-template

StateDD werkt omdat deze bestanden samen één gedeelde projectwaarheid vormen.



Rol per file

File	Rol	Wie gebruikt het?	Wanneer updaten?
AGENTS.md	Operating contract	Elke agent	Zelden, bij proceswijziging
STATUS.md	Menselijke snapshot	Human / CTO / teamlead	Elke sessie
PROJECT_STATE.yaml	Machine-checkable truth	Agents + scripts	Wanneer waarheid verandert
PROJECT_DNA.yaml	Stabiele architectuur	CTO + coding agents	Bij architectuurbeslissingen
NEXT_ACTIONS.md	Actieve queue	CTO + coding agent	Elke sessie
BACKLOG.md	Roadmap met stable IDs	CTO / team	Bij scope / roadmap wijziging
EVIDENCE_LOG.md	Bewijsledger	CTO / reviewer / evidence	Bij elk bewijsartefact



Waarom **niet** zomaar files overslaan?

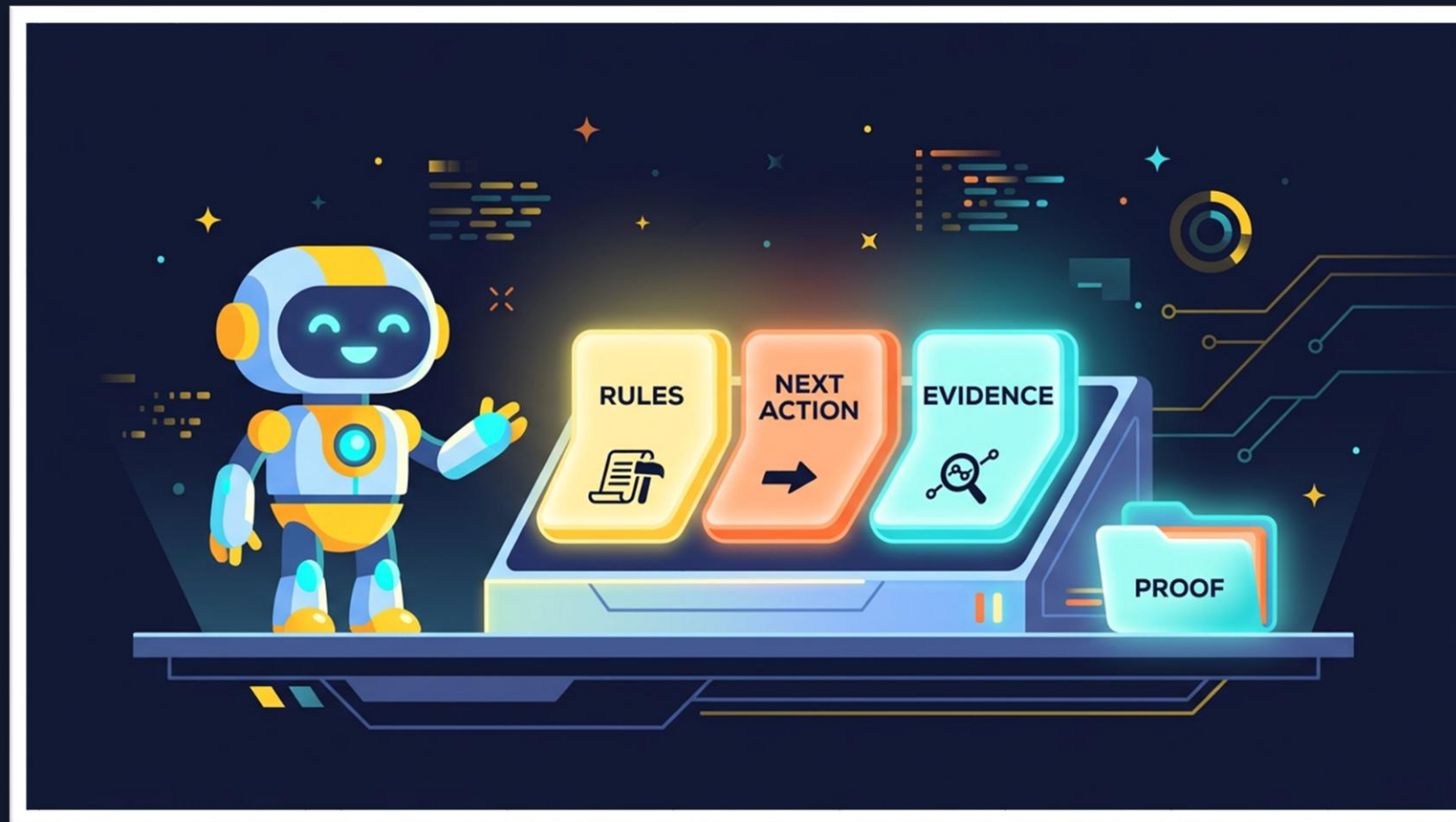
Eén ontbrekende state-laag wordt later vaak context decay.

- Zonder **PROJECT_DNA.yaml** verliest de agent architectuurcontext
- Zonder **PROJECT_STATE.yaml** is current truth niet machine-checkable
- Zonder **BACKLOG.md** verlies je stable IDs en scope
- Zonder **NEXT_ACTIONS.md** weet de agent niet wat nu telt
- Zonder **EVIDENCE_LOG.md** worden claims niet auditable
- Zonder **STATUS.md** wordt menselijke overdracht traag
- Zonder **AGENTS.md** verzint elke agent zijn eigen regels



Begin klein, maar gebruik de **volledige template**

Vanaf dag één: **alle 7 files** aanwezig.



- Start vanuit de volledige StateDD_Template
- Alle state files hebben een rol
- Begin klein in gedrag: één slice, één bundle, één handoff
- Niet klein in waarheid: geen ontbrekende laag
- Voorkomt dat elke agent zijn eigen proces verzint



03.

Van CTO-beslissing naar **Werkcontract**

De CTO vertaalt doel naar controleerbare opdracht



De **CTO** maakt het werkcontract

De coding agent krijgt geen wens.

Hij krijgt een **controleerbaar werkcontract**.

- Productdoel wordt vertaald naar **scope**
- Risico's worden vertaald naar **out-of-scope**
- "Klaar" wordt vertaald naar **acceptance criteria**
- Tools worden gekoppeld aan **bewijs**
- De coding agent krijgt **geen open einde**



Welke rol **speel** jij?

In StateDD is de agent **uitvoerder**. Niet rechter, architect en product owner tegelijk.



Product Owner

- Bepaalt wat waardevol is
- Formuleert probleem en doel
- Beslist over richting en prioriteit



CTO / Reviewer

- Vertaalt doel naar werkcontract
- Bewaakt scope, architectuur en bewijs
- Beslist accepted / conditional / rejected



Coding Agent

- Voert uit binnen scope
- Gebruikt tools, levert proof bundle
- Keurt zichzelf **niet** goed



Slechte prompt vs Werkcontract BL-002

Slecht

"Bouw een AI-kloon app"

- Geen scope
- Geen consent
- Geen grenzen
- Geen verificatie
- Geen governance

Werkcontract BL-002

"Implementeer BL-002: Consent gate"

- Scope: form + selector + blocking + audit log
- Out-of-scope: voice, face, scraping, memory
- Acceptance: block unauthorized, warning visible
- Tests pass, browser proof



Scope, Out-of-Scope & Acceptance Criteria

Scope

- Consent type selector
- Blocking validation
- Audit log entry

Out-of-Scope

- Voice cloning
- Face generation
- Scraping
- Persistent memory
- External APIs

Acceptance Criteria

- Block unauthorized
- Warning visible
- Tests pass
- Browser proof
- Proof bundle +
worktree clean



Closure Gates

Elke sessie eindigt pas als **alle gates expliciet beantwoord** zijn.

- ✓ Geen ongerelateerde changes in diff
- ✓ Geen "klaar" zonder bewijs
- ✓ Tests geslaagd en gedocumenteerd
- ✓ Runtime identity bewezen
- ✓ Worktree clean bij handoff



04.

Subagents & MCP

Waarom één agent niet alles zelf mag doen

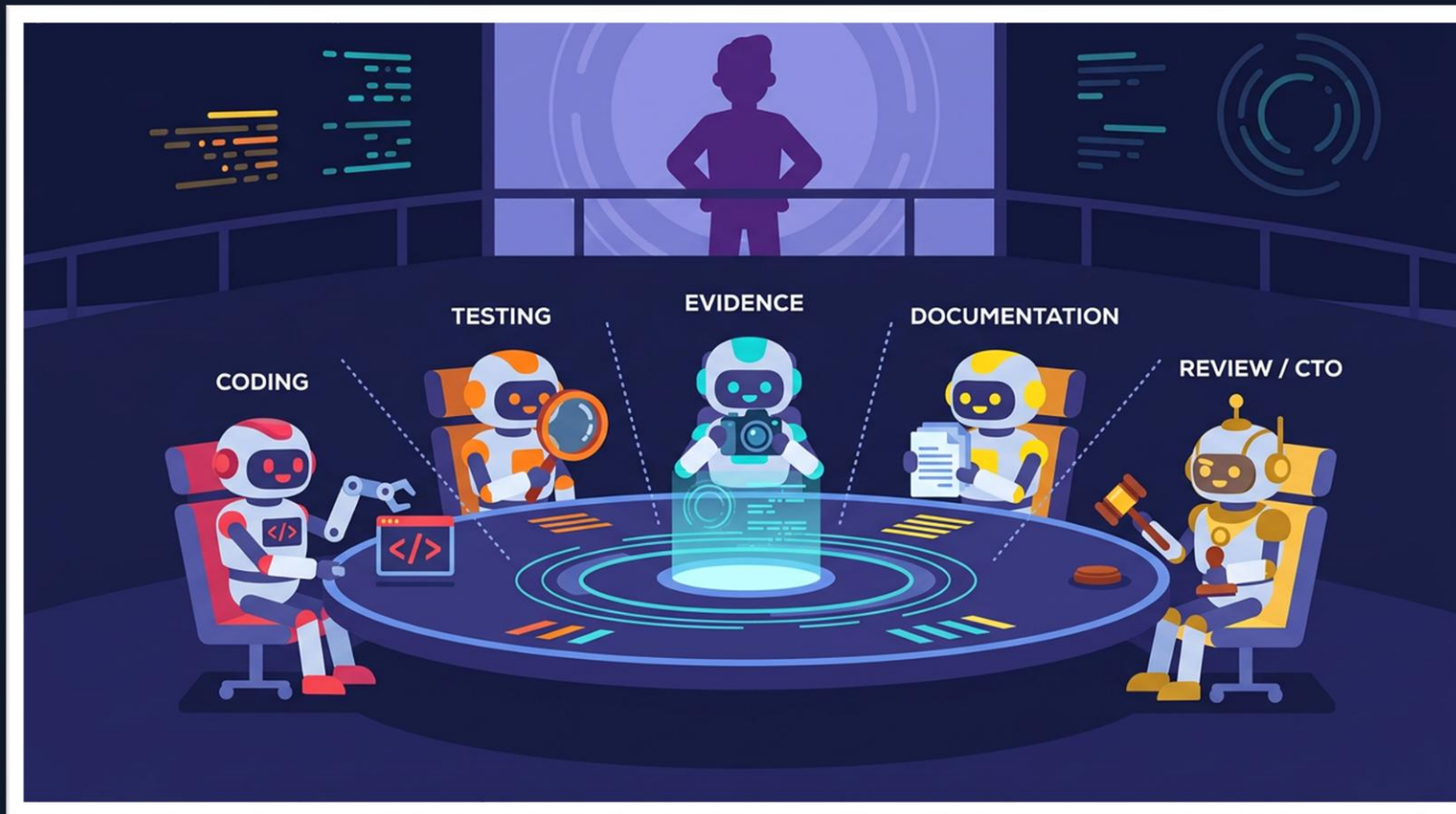


04. Subagents kunnen gebruikt worden om meer werk te verrichten per sessie

Hoofd coding agent maakt subagents en controleert hun werk.



Subagents: **parallele uitvoering**



Uitvoerende agents worden gemaakt door hoofd coding agent -> Agent Swarm principe

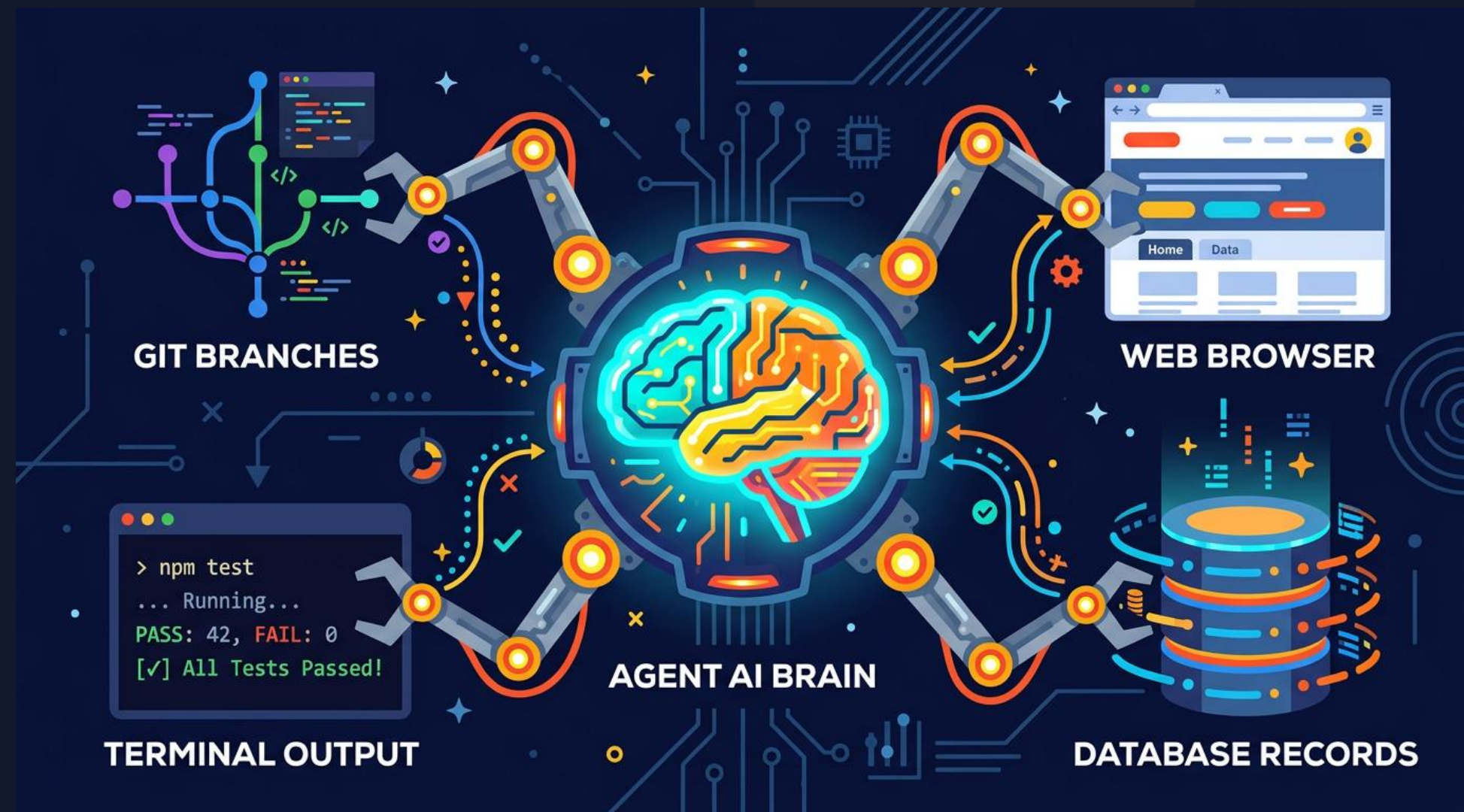


MCP: **tools** in plaats van **gokken**

MCP verbindt agents met **echte output**.

Agent denkt → Tool gebruiken → Echte output zien →
Volgende beslissing

- **Git**: welke branch, welke diff?
- **Terminal**: slagen tests echt?
- **Browser**: werkt de UI echt?
- **Database**: klopt de data echt?



04.

9 MCP Types



Filesystem

Read, write, list files



Git

Branch, diff, commit, status



Terminal

Run tests, build, scripts



Browser / CDP

Navigate, click, screenshot



Screenshot

Capture UI as proof artifact



Database

Query, inspect, verify data



Issue Tracker

Create, update, link tickets



Docs / Export

Generate reports and docs



Secrets Scanner

Detect leaks in code



04.

Niet elke agent krijgt elk gereedschap

MCP zonder permissies is chaos versnellen.

Agent	Filesystem	Git	Terminal	Browser	Database	Review
Coding	Write	Read	Tests	Optional	No prod	No
Test	Read	Read	Yes	Yes	Read-only	No
Evidence	Read	Read	Logs	Screenshots	No	No
Doc	Write docs	Read	No	No	No	No
CTO	Read	Read	Optional	Review	No prod	Yes



05.

Proof Bundles

No proof, no closure



Chat claim vs Proof Bundle

Chat Claim

- Verdwijnt bij nieuwe sessie
- Geen reproduceerbaarheid
- Agent claimt zonder bewijs
- Geen audit trail
- Niet reviewbaar door anderen

Proof Bundle

- Genummerde folder
- Git-tracked
- Reviewbaar door CTO
- Audit trail ingebouwd



De proof bundle is de **overdracht**

Elke sessie krijgt een nieuwe genummerde evidence folder.

```
docs/evidence/  
session-001-bl-002-consent-gate/  
00-HANDOFF.md  
01-PROOF_MANIFEST.yaml  
02-SCOPE.md  
03-CHANGED_FILES.md  
04-TEST_RESULTS.md  
05-BROWSER_PROOF.md  
06-GIT_STATUS.txt  
07-KNOWN_LIMITATIONS.md  
screenshots/  
logs/
```

PersonaLab BL-002

De nummering is bewust. Het forceert volgorde.

Review agent weet:

begin bij 00, werk naar 07

Geen losse files. Geen ad-hoc bewijs. Alles in de bundle.



05.

"No proof, no closure"

Geen tests? Niet klaar.

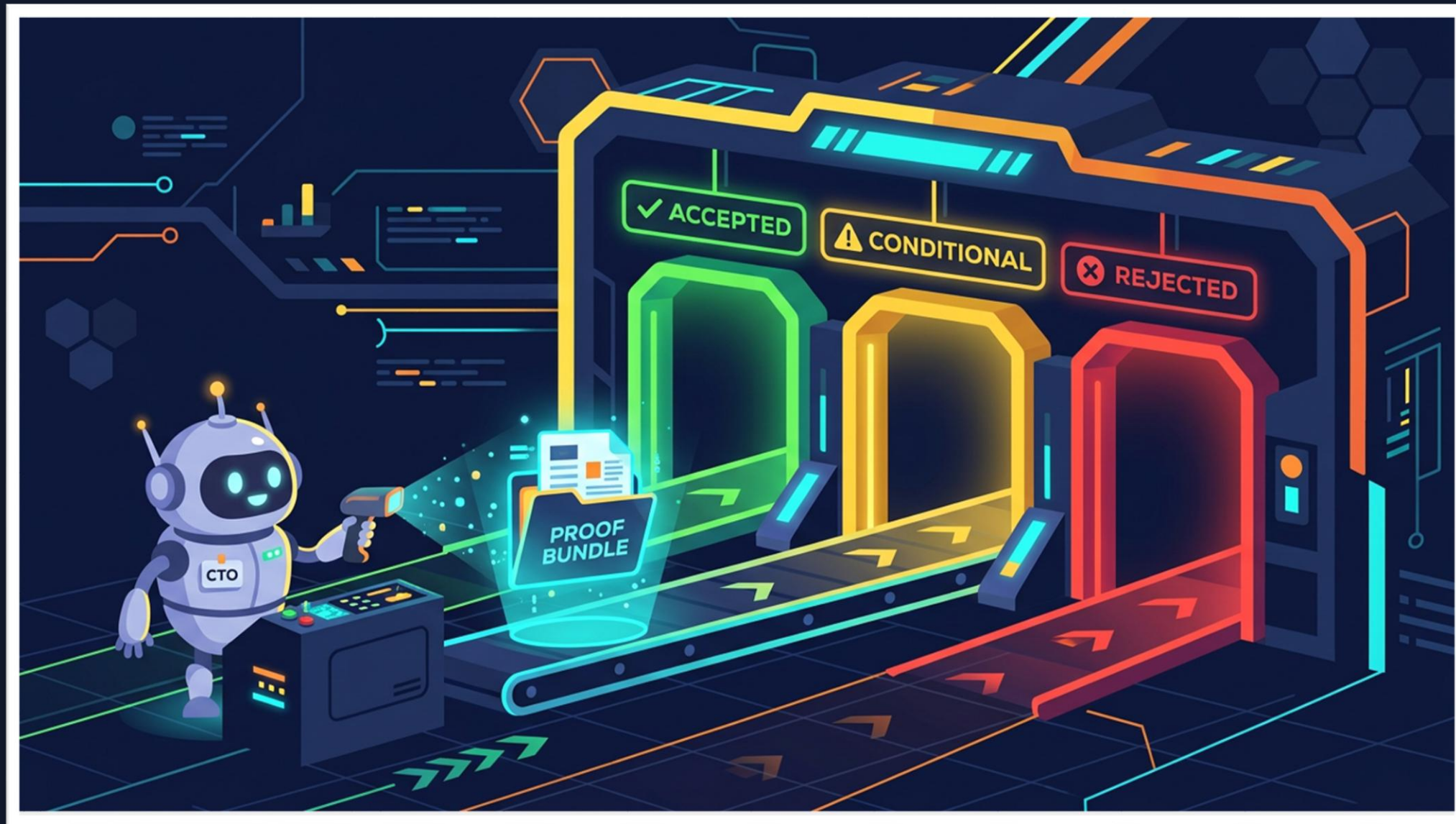
Geen browser proof? Niet bewezen.

Dirty worktree? Niet overdraagbaar.

Geen state update? Context gaat verloren.



CTO Review: drie mogelijke uitkomsten



ACCEPTED

Alles bewezen, state bijgewerkt,
worktree clean

CONDITIONAL

Implementatie lijkt goed, maar bewijs
of docs missen deels

REJECTED

Scope fout, bewijs ontbreekt, runtime
onduidelijk, dirty worktree



Demo: **PersonaLab BL-002** in 7 stappen

- 1 **CTO kiest** BL-002: Consent gate
- 2 **CTO schrijft** contract met scope + acceptance
- 3 **Coding agent bouwt** form + blocking + audit log
- 4 **Test agent runt** validation tests
- 5 **Browser proof:** fictional OK + real-person blocked + warning visible
- 6 **Proof bundle:** session-001-bl-002-consent-gate/
- 7 **CTO review** → Accepted / Conditional / Rejected



06.

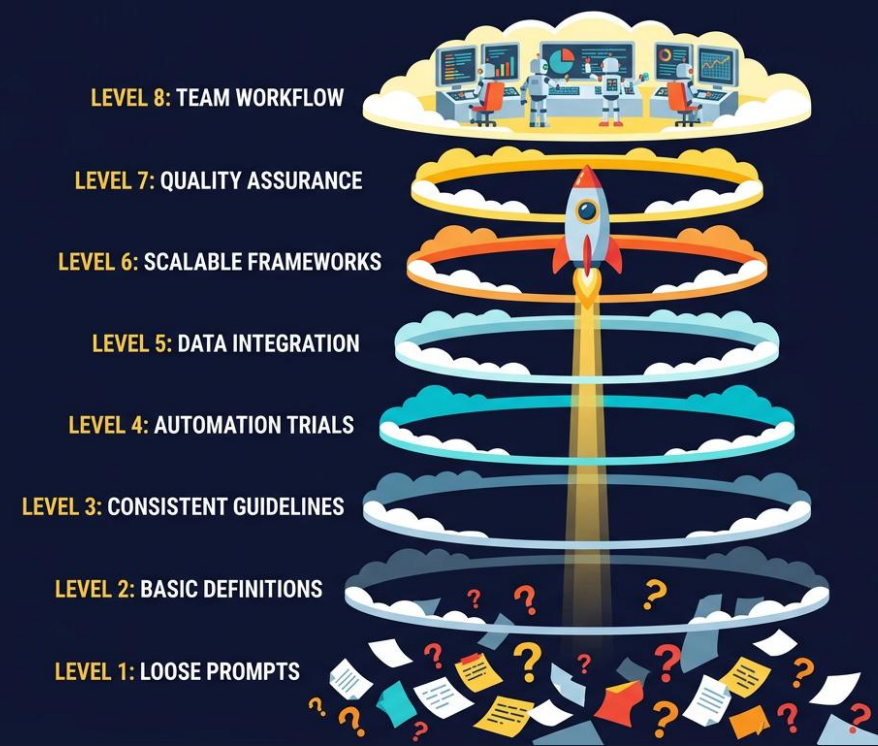
Workshop & Maturity

Van niveau 0 naar niveau 7



06.

Maturity Ladder



0. Losse prompts

1. Betere prompt

2. State files

3. Tools & verificatie

4. Proof bundle

5. CTO review

6. Closure gates

7. Team workflow

Je hoeft niet direct naar niveau 7. De meeste teams beginnen bij 2-3 en bouwen geleidelijk op.



06.

Agenda: Halve dag (3.5 uur)

00:00–00:20 Opening + herkenbare failures

00:20–01:00 Context decay, fake progress, runtime confusion

01:00–01:40 StateDD als project-OS

01:40–01:50 *Pauze*

01:50–02:30 CTO-beslissing → werkcontract

02:30–03:00 Subagents, MCP, proof bundles

03:00–03:25 Mini-oefening: beoordeel CTO-werkcontract

03:25–03:30 Takeaways



06.

Workshop: Jij bent de CTO-reviewer

Deelnemers leren AI-output **professioneel beoordelen**.

Je krijgt: PersonaLab BL-002, een volledige StateDD-template, een CTO-werkcontract en een proof bundle.

1. Is de opdracht correct afgebakend?
2. Zijn alle relevante StateDD-files gebruikt?
3. Is het bewijs voldoende?
4. Zijn er ontbrekende closure gates?
5. Wat is je beslissing: **ACCEPTED**, **CONDITIONAL** of **REJECTED**?
6. Wat is de volgende NEXT_ACTION?



06.

Begin met één regel

Geen claim zonder bewijs.

- Je hoeft geen perfect systeem te bouwen
- Je hoeft niet alles tegelijk te doen
- Begin met één regel, één week, één team
- Daarna pas de volgende trede



06.

De toekomst is niet: **betere prompts.**
De toekomst is: **betere workflows rond agents.**

- StateDD is geen prompttruc
- Het is een herbruikbaar project-OS voor AI-driven development
- De toekomst is dat teams betere workflows bouwen rond agents



Vragen?

github.com/lennertvhoy/StateDD_Template

Repo open source, MIT licensed
Gebruik het, fork het, verbeter het



Coding-Agent Prompt

Deze prompt wordt normaal opgesteld door de CTO/human of CTO-agent.

=== CODING AGENT — WERKCONTRACT ===

IDENTITEIT: Jij bent een coding agent. Voer alleen uit wat hier staat.

READ ORDER — lees in deze volgorde:

1. AGENTS.md → Operating contract / regels
2. STATUS.md → Menselijke snapshot
3. PROJECT_STATE.yaml → Machine-checkable current truth
4. PROJECT_DNA.yaml → Stabiele architectuur / governance
5. NEXT_ACTIONS.md → Actieve queue
6. BACKLOG.md → Roadmap met stable IDs
7. EVIDENCE_LOG.md → Bewijsledger

OPDRACHT: [BL-002 Consent Gate]

Scope: form + selector + blocking + audit log

Out: voice cloning, face gen, scraping, memory

Accept: tests pass, browser proof, worktree clean

REGELS:

- Geen change zonder test
- Geen "klaar" zonder proof bundle
- Geen refactoring buiten scope
- Worktree clean bij handoff
- Keur jezelf niet goed — wacht CTO review



CTO-Review Checklist

De CTO beoordeelt, niet de coding agent.

=== CTO REVIEW — 11-PUNT CHECKLIST ===

1. **SCOPE-CHECK** → Alleen geplande changes in diff?
2. **DIFF-REVIEW** → Geen ongerelateerde files gewijzigd?
3. **TEST-RESULTATEN** → Alle relevante tests geslaagd?
4. **RUNTIME IDENTITY** → Screenshots tonen juiste URL + versie?
5. **PROOF BUNDLE** → Alle 00-07 bestanden aanwezig?
6. **STATE UPDATE** → PROJECT_STATE en STATUS bijgewerkt?
7. **BACKLOG SYNC** → BL-ID correct gesloten of voortgezet?
8. **WORKTREE CLEAN** → Geen ongecommitte of ongetrackte changes?
9. **KNOWN LIMITATIONS** → Beperkingen expliciet gedocumenteerd?
10. **SECURITY CHECK** → Geen secrets gelekt, geen scope overreach?
11. **NEXT_ACTIONS** → Duidelijke vervolgstap geformuleerd?

BESLISSING:

ACCEPTED → Alles bewezen, state clean, door naar volgende

CONDITIONAL → Implementatie OK, bewijs/docs deels missen

REJECTED → Scope fout, bewijs ontbreekt, runtime onduidelijk



Proof Manifest

=== PROOF MANIFEST — 01-PROOF_MANIFEST.yaml ===

session_id: session-001-bl-002-consent-gate

backlog_id: BL-002

status: ACCEPTED | reviewer: CTO-Agent

timestamp: 2025-01-15T14:32:00Z

files:

- 00-HANDOFF.md
- 01-PROOF_MANIFEST.yaml
- 02-SCOPE.md
- 03-CHANGED_FILES.md
- 04-TEST_RESULTS.md
- 05-BROWSER_PROOF.md
- 06-GIT_STATUS.txt
- 07-KNOWN_LIMITATIONS.md

screenshots:

- 001-fictional-persona-created.png
- 002-real-person-blocked.png
- 003-warning-visible.png

test_result: PASS | browser_proof: 3 screenshots

worktree_clean: true



Closure Gates

Elke sessie eindigt pas als alle gates expliciet beantwoord zijn.

1. Geen ongerelateerde changes in diff
2. Geen "klaar" zonder bewijs
3. Tests geslaagd en gedocumenteerd
4. Runtime identity bewezen (screenshots + URL)
5. Worktree clean bij handoff
6. State files bijgewerkt (PROJECT_STATE + STATUS)
7. Proof bundle compleet (00-07 + screenshots)
8. Known limitations gedocumenteerd



Proof Bundle Folder Tree

=== PERSONALAB BL-002 — PROOF BUNDLE ===

docs/evidence/session-001-bl-002-consent-gate/

- |—— 00-HANDOFF.md
- |—— 01-PROOF_MANIFEST.yaml
- |—— 02-SCOPE.md
- |—— 03-CHANGED_FILES.md
- |—— 04-TEST_RESULTS.md
- |—— 05-BROWSER_PROOF.md
- |—— 06-GIT_STATUS.txt
- |—— 07-KNOWN_LIMITATIONS.md
- |—— screenshots/
 - | |—— 001-fictional-persona-created.png
 - | |—— 002-real-person-blocked.png
 - | |—— 003-warning-visible.png
- |—— logs/
 - | |—— test-run-2025-01-15.log
- |—— attachments/
- |—— runtime-config.json

Totaal: 8 bewijsbestanden + 3 screenshots + 1 log + 1 config

